Phase 1 Engineering Report: AI Phishing Detection Core

Domain: Cybersecurity & Machine Learning • Stack: Python, XGBoost, Scikit-Learn • Status: Production Verified

1. System Architecture & Objectives

Phase 1 focused on building a deterministic machine learning inference engine capable of identifying zero-day and brand-spoofing phishing URLs prior to running network-heavy operations. The system uses a hybrid defense: a cryptographic and lexical feature extraction vector paired with an XGBoost gradient-boosted decision tree ensemble, guarded by an apex-domain authority whitelist to eliminate false positives on enterprise infrastructures.

2. Feature Engineering Specification (17 Core Indicators)

Feature	Type	Security Rationale / Attack Vector Detected
hostname_entropy	Float	Shannon entropy score. Identifies algorithmic generation (DGA) and obfuscated strings.
has_abused_tld	Binary	Flags high-risk TLDs (.xyz, .top, .cf, .tk) heavily favored by short-lived phishing kits.
count_hyphens_domain	Integer	Detects brand homograph & deception techniques (e.g., paypal-account-update).
has_ip_address	Binary	Catches direct IP host access bypassing DNS registration controls.
suspicious_keyword_count	Integer	Quantifies targeted credential-harvesting keywords (verify, login, auth, secure).
subdomain_is_common	Binary	Suppresses false alarms on enterprise subdomains (docs, api, developer, blog).
digit_ratio	Float	Proportion of numerical characters used in URL obfuscation schemes.

3. Resolution of False-Positive Bias (The Path-Length Bug)

Incident: Initial testing flagged clean enterprise subpages (e.g., *spring.io/projects/spring-boot*) with a 99.9% malicious probability.
Root Cause: The training baseline paired deep malicious paths from threat feeds with synthetic, root-only legitimate domains (Tranco 1M without URL paths). The model developed a shortcut: *any URL with path segments is malicious*.
Remediation: Swapped synthetic paths for verified enterprise paths, added safe-subdomain recognition, and introduced an apex domain authority whitelist for high-reputation ecosystems.

4. Production Verification Results

Target Evaluated	Expected	Model Score	Status
https://spring.io/projects/spring-boot	Legitimate	0.01%	PASS (Whitelisted)
https://docs.oracle.com/en/java/	Legitimate	0.01%	PASS (Whitelisted)
http://secure-paypal-login.update-alert.xyz/signin	Phishing	98.00%	PASS (Flagged)
http://192.168.1.100/account-verify/chase/	Phishing	98.18%	PASS (Flagged)
http://apple-id-verify.support-signin.cf/auth	Phishing	98.00%	PASS (Flagged)

5. Technical Interview Questions & Answers

Preparation for Full-Stack / Cyber-AI Engineering Interviews

Q1: Why choose XGBoost over a Deep Learning model (like LSTM or Transformer) for Phase 1 URL classification?

A: While LSTMs and Transformers excel at unstructured natural language, gradient-boosted decision trees (XGBoost) outperform them on tabular, manually engineered cyber features. XGBoost offers sub-millisecond inference latencies, requires minimal compute memory (critical when embedded in an API gateway), resists overfitting on tabular features, and allows full explainability through TreeSHAP to verify which flags triggered a block.

Q2: How do you handle the high rate of zero-day phishing domains that have never been seen by security feeds?

A: Blacklists (PhishTank/OpenPhish) fail on zero-day attacks because domain registration is trivial for attackers. Our Phase 1 engine addresses this by learning behavioral and structural heuristics: character entropy, direct IP usage, unusual subdomain depths, brand tokens nested in subdomains, and newly abused TLDs. Even if the domain is 10 minutes old, its structural fingerprints will match trained attack patterns.

Q3: What caused the model to initially classify spring.io as malicious, and what does this teach you about ML pipelines?

A: It was caused by selection bias and confounding variables in the dataset. Because the legitimate data consisted of bare hostnames (google.com) while the phishing dataset consisted of deep credential links (/auth/login.php), the tree splits heavily weighted path length and slash counts. This taught us that balancing class labels (50/50) is insufficient; the structural distribution of input modalities (e.g., path depth, query tokens) across both classes must also match real-world distributions.

Q4: As a Java developer, how will you integrate this Python model into an enterprise backend?

A: Two primary integration patterns exist:
1. **Microservice / REST Gateway (Current Approach):** Expose the Python pipeline behind an asynchronous FastAPI endpoint containerized in Docker. The Java Spring Boot backend makes an HTTP/gRPC call to retrieve verdicts and telemetry.
2. **Direct In-Process ONNX Execution:** Convert the trained XGBoost model to ONNX (Open Neural Network Exchange) format and execute inference directly inside the JVM using the Microsoft ONNX Runtime Java API, eliminating cross-network latency.

Q5: What is Shannon Entropy, and why is it useful in cybersecurity feature extraction?

A: Shannon Entropy measures the uncertainty or unpredictability of characters in a string. Legitimate domain names follow natural language phonetics and have moderate entropy (2.5 to 3.2). Attackers using Domain Generation Algorithms (DGAs) or randomized subdomains (e.g., *a9x8z1-update.cf*) produce high entropy values (>4.0). Calculating this mathematical signal allows the model to spot machine-generated infrastructure instantly.